GIT AND GITHUB

What will we cover?

We will start with the prerequisites, then use Git and GitHub to save and share code.

1. **Files and folders**
   Files, folders, paths, and VS Code
2. **Git and GitHub**
   Repositories, commits, and history
3. **Create a repository**
   GitHub Desktop and publishing online
4. **Use the terminal**
   Clone, pull, add, commit, and push

> **Goal:** create a Python file, save it with Git, and publish it on GitHub.

PREREQUISITES

What is a file?

A file is **information stored on a computer and given a name**, so you can find it and use it again later.

INSIDE A FILE

Files are made of bits

A **bit** is the smallest piece of computer information. It can be either `0` or `1`.

A file contains many bits. Their pattern can represent words, pictures, sounds, or instructions.

WHAT YOU CAN DO

You work with files using apps

You can open, change, save, copy, move, share, or delete a file.

The file stays on your computer until you delete it.

> `hello.txt` is a file name. `hello` is the name you choose; `.txt` is the **file extension**. It indicates that this is a text file and helps the computer choose an app to open it.

PREREQUISITES

Text vs Binary files

TEXT

`hello.txt`

```
Hello, Ada!
  Have a wonderful day.
```

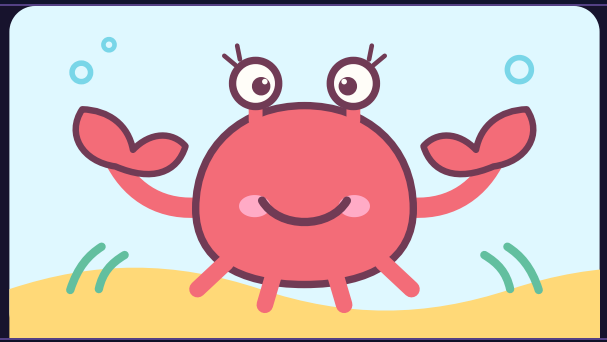
A text editor like Visual Studio Code shows the file's characters and lets you edit them.

BINARY

`crab.png`

Can be opened with an image app like Photos

The app displays the picture.



PREREQUISITES

What is a folder?



It organizes data

A folder holds files and other folders.

It has a name

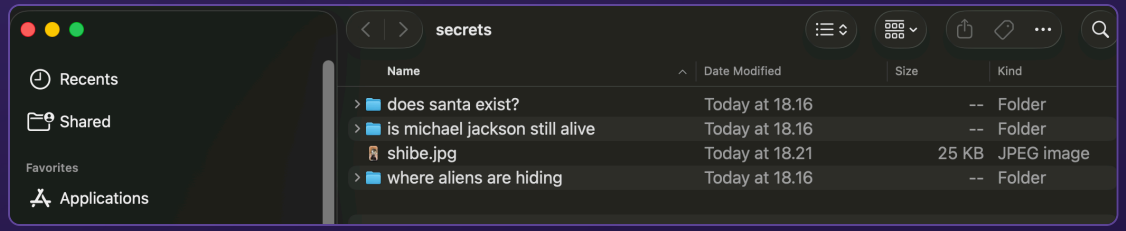
The name helps people and programs find its contents.

It is also called a directory

The two words mean the same thing. In this lesson, we will use folder.

PREREQUISITES

Example: a folder



secrets is the folder currently open in Finder.

It contains three folders and one image file.

> The **icons in the image** help us tell them apart: folders have folder icons, while the image file shows a picture thumbnail.

PREREQUISITES

Directories form a tree

```
~
|-- Documents/
|   `-- Projects/
|       `-- my-website/
|           |-- README.md
|           `-- index.html
`-- Downloads/
```

Parent

The folder one level above.

Child

A file or folder inside another folder.

Tree

The whole branching structure.

> Start at the top, then follow one branch to find a file.

PREREQUISITES

Your user folder

ALEX

Every user account has a personal folder for its files and projects.

Windows	`C:\Users\alex`
macOS	`/Users/alex`

> Your user folder usually contains **Desktop**, **Documents**, and **Downloads**.

PREREQUISITES

A PATH IS AN ADDRESS

A path tells the computer exactly where a file or folder lives.

`C:` \ `Users` \ `alex` \ `Documents` \ `photos` \ **`crab.png`**

Start

`C:` is the drive.

Follow the folders

Open each folder from left to right.

Arrive

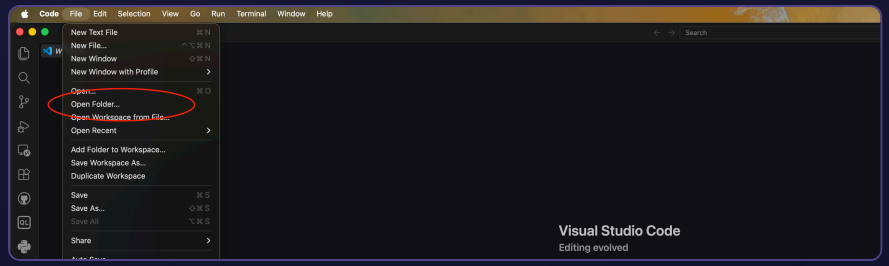
`crab.png` is the file.

> On macOS, the same idea uses `/` instead: `/Users/alex/Documents/photos/crab.png`

VS CODE · STEP BY STEP

Opening a project and creating a file

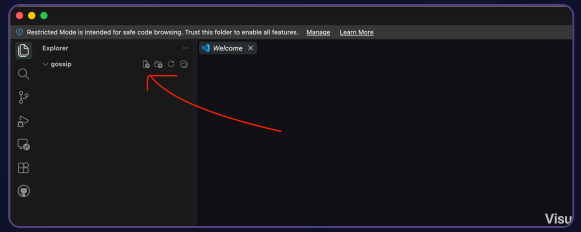
1 In VS Code, choose **File → Open Folder...**



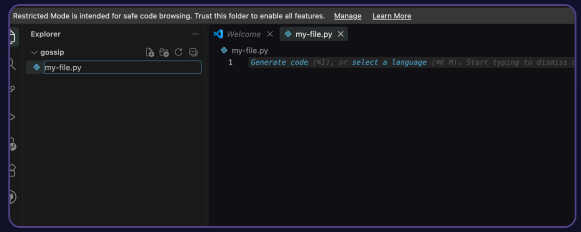
2 Select your project folder. Click **Open** on macOS or **Select Folder** on Windows.



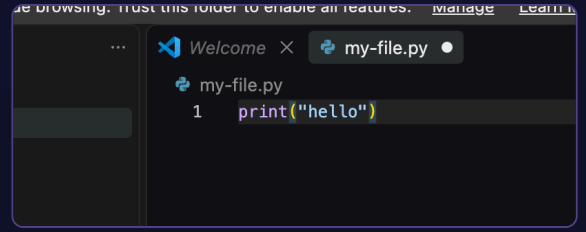
3 In Explorer, click **New File**.



4 Type `my-file.py`, then press **Enter**.



5 Type `print("hello")`, then save.



GITHUB DESKTOP · STEP BY STEP

Create your first repository

Make a project on your computer, publish it to GitHub, and open it in Visual Studio Code.

1 Create → 2 Name → 3 Publish → 4 Open

BEFORE WE CREATE A REPOSITORY

Git and GitHub.com

01

Git

A tool on your computer that records changes to a project. It lets you see what changed and return to an earlier version.

02

GitHub

A website that stores Git projects online. It lets you share your work and collaborate with other people.

How will we use them? Git keeps the history, and GitHub keeps an online copy. We will create and publish the repository with GitHub Desktop, then use Git by typing commands in VS Code's terminal.

A NEW WORD

What is a repository?

A **repository**, or **repo**, is a project folder that Git keeps track of.

YOUR PROJECT

The files you work on

It contains the code, images, notes, and other files that belong to one project.

GIT

A history of changes

Each saved point in the project's history is called a **commit**. Commits show what changed, who changed it, and when.

> A repository can be on your computer, on GitHub.com, or in both places.

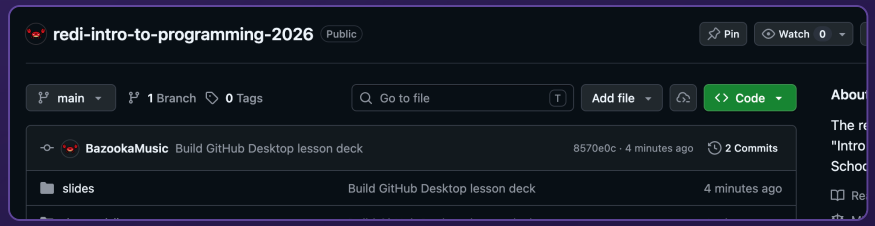
SEE A REPOSITORY ONLINE

Where are the commits?

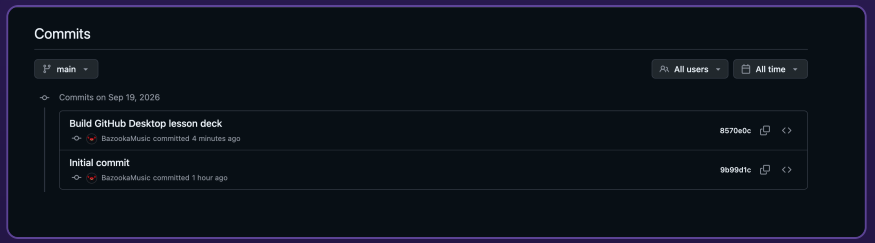
Our course repository

https://github.com/BazookaMusic/redi-intro-to-programming-2026

1. Open the repository and click the commits link.



2. GitHub shows the project's commit history.



STEP 1 OF 4

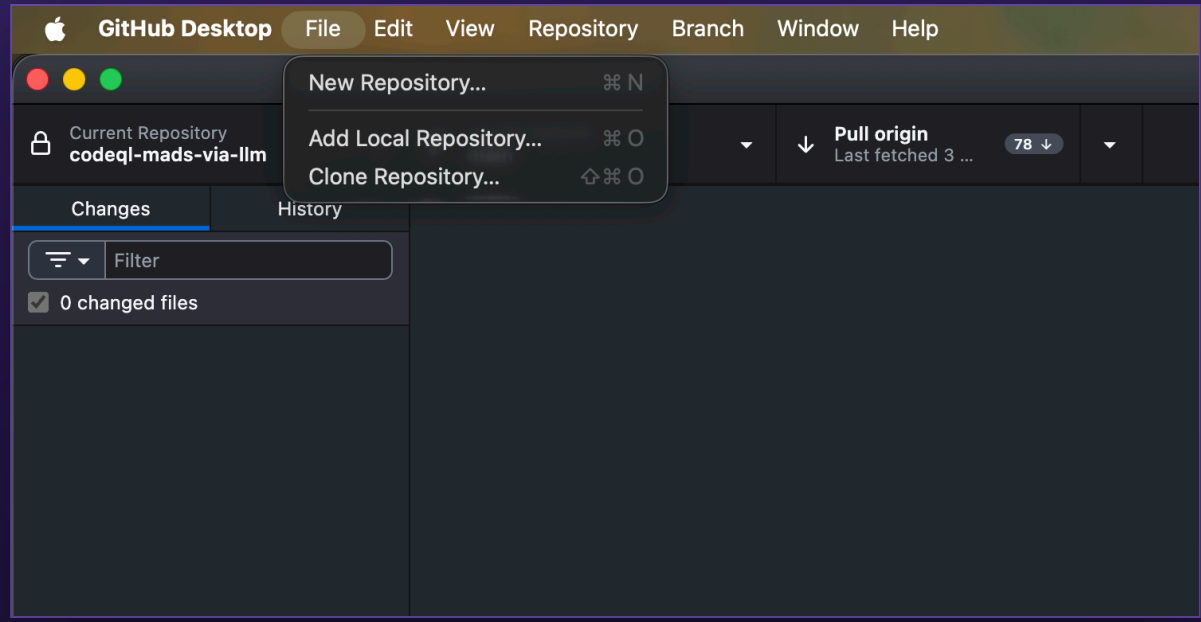
Start a new repository

FIRST

Open GitHub Desktop.

Then choose File → New Repository...

> A repository is the project folder Git will track.



STEP 2 OF 4

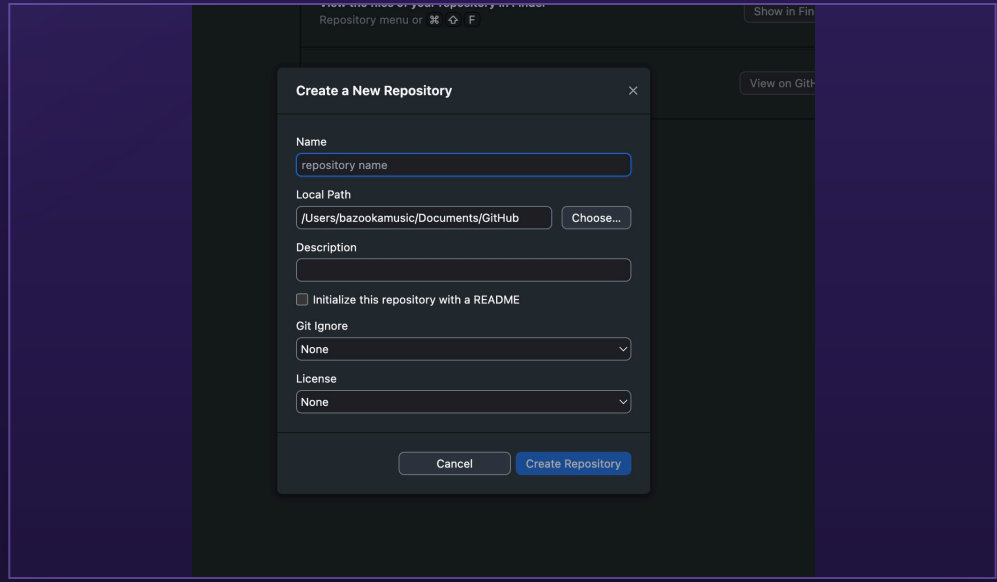
Choose the repository details

1. Give the repository a short, clear name.

2. Choose where to save it.

3. Tick the box to initialize with a readme.

Click Create Repository.



PAUSE AND CHECK

Your repository now exists locally

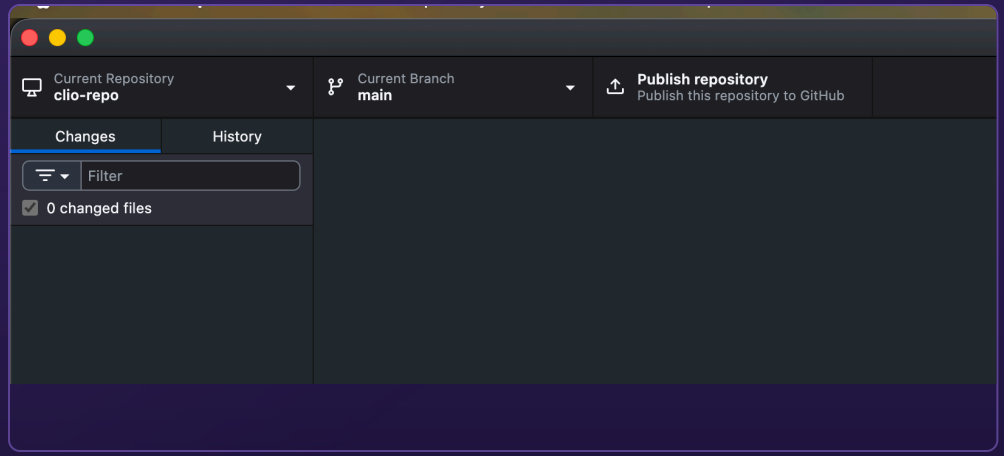
GitHub Desktop created the project folder on your computer.

✓ Folder created ✓ Git history started ○ Not online yet

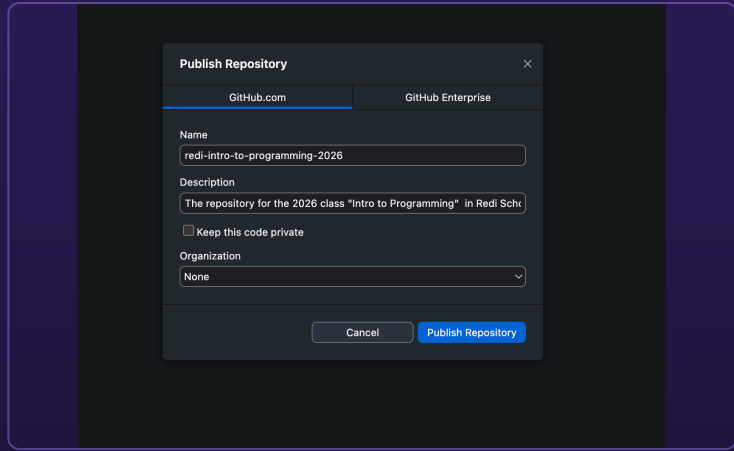
STEP 3 OF 4

Publish it to GitHub

1. Click Publish repository in the top bar.



2. Review the name, description, and visibility. Then click Publish Repository.



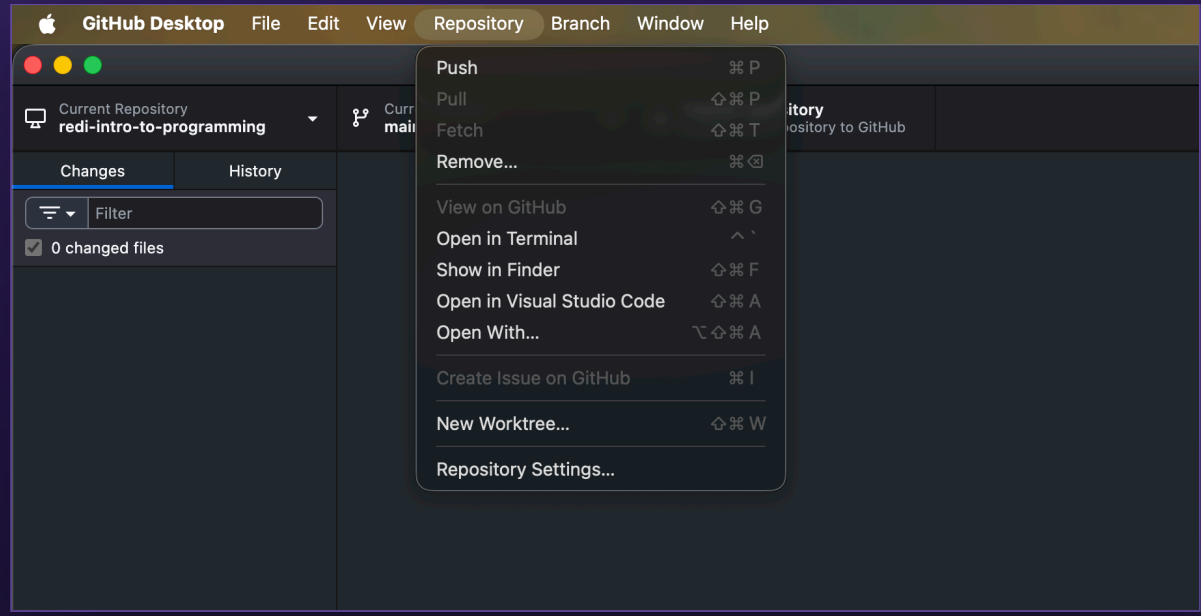
STEP 4 OF 4

Open the project in VS Code

OPEN

Repository → Open in Visual Studio Code

> Now you can create files and begin coding inside the repository.

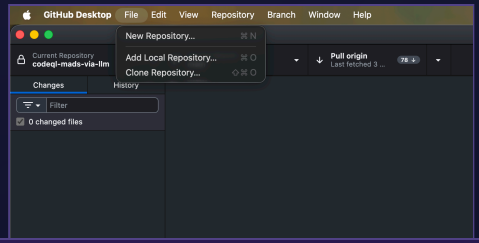


EXERCISE

Create and publish your repository

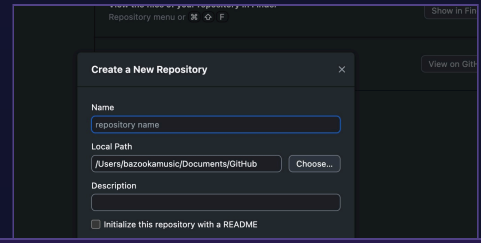
1 Create

File → New Repository...



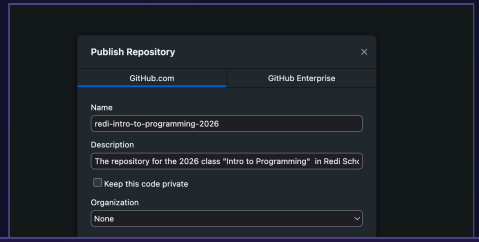
2 Set up

Name it, choose a folder, and add a README.



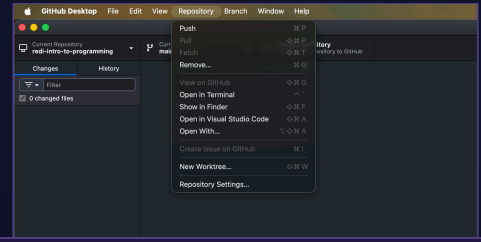
3 Publish

Click **Publish repository**.



4 Open

Repository → Open in Visual Studio Code



Done when: your repository is online and open in VS Code.

REPOSITORY READY

You made it

1 Created locally 2 Published online 3 Opened in VS Code

Open your repository in your browser (e.g. Chrome, Edge, or Safari) at:

```
www.github.com/username/name-of-your-repo
```

Replace username with your GitHub username and name-of-your-repo with your repository name.

Example: **www.github.com/bazookamusic/redi-example**

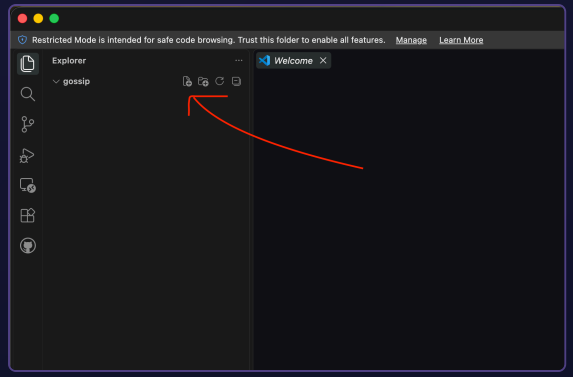
Next: create a file inside your repository.

YOUR TURN · VS CODE

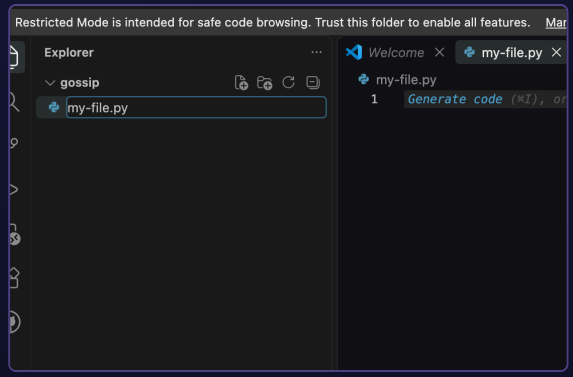
Create a file in your repository

Your repository is already open in VS Code. Now add your first Python file.

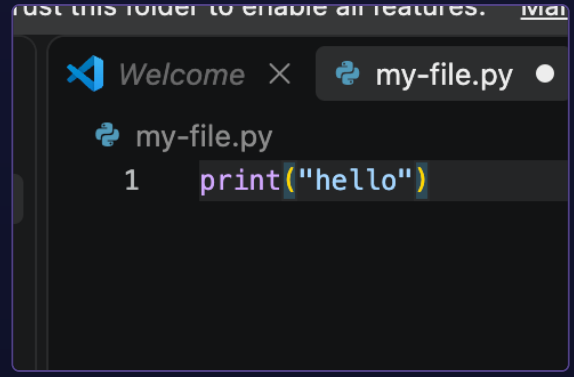
1. In Explorer, click **New File**.



2. Type `my-file.py`, then press **Enter**.



3. Type `print("hello")`, then save.



Done when: `my-file.py` appears inside your repository folder.

BEFORE TYPING COMMANDS

What is the terminal?

The terminal is an app where you **write commands to run programs** and control your computer.

RUN A PROGRAM

Write a command

Windows:

```
python hello.py
```

macOS:

```
python3 hello.py
```

Press **Enter**. The terminal returns text called **output**, describing what the command did or showing the program's result.

WORKING FOLDER

Commands run in one folder

The terminal has a **working folder**: the folder where it currently runs commands.

`$HOME` means your current user's home folder.

Go to a specific folder on macOS:

```
cd "$HOME/Documents/my-project"
```

On Windows PowerShell:

```
cd "$HOME\Documents\my-project"
```

Go up to the parent folder:

```
cd ..
```

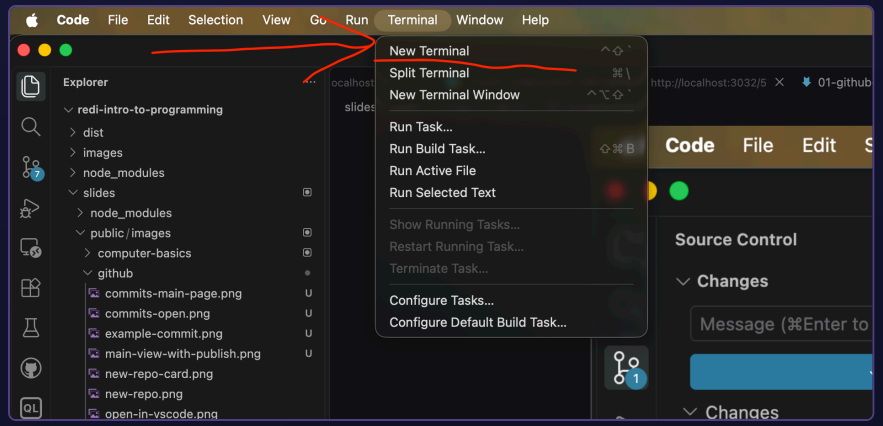
> **Open it:** On Windows, search for **Terminal** or **PowerShell**. On macOS, open **Applications → Utilities → Terminal**. In VS Code, **Terminal → New Terminal** works the same on both.

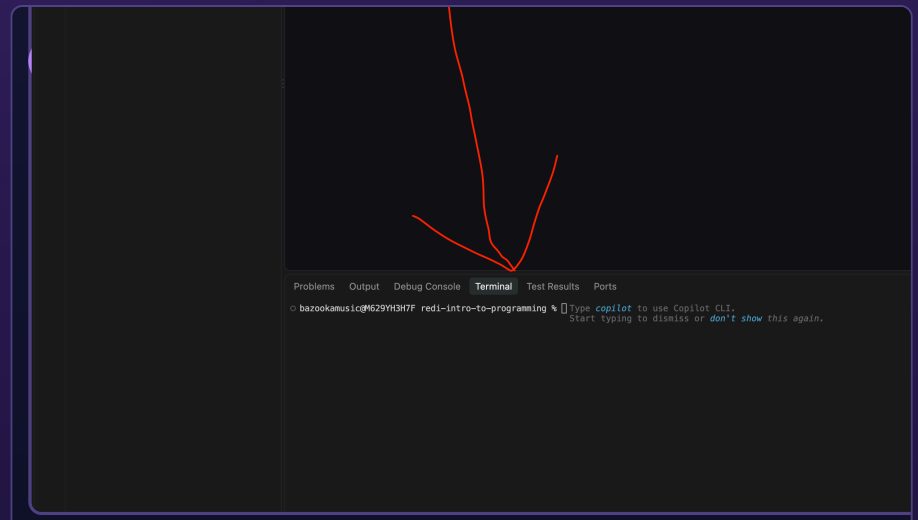
VS CODE · STEP BY STEP

Open the terminal

The terminal lets you give text commands to your computer without leaving VS Code.

1 Choose **Terminal → New Terminal**.





Check: the terminal shows the name of your repository next to the prompt.

TERMINAL · SAVE AND PUBLISH

1. Check and stage the file

Type or copy one command at a time. **Press Enter after each command** to run it.

Check what Git sees

```
git status
```

```
On branch main
Your branch is up to date with 'origin/main'.
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  hello.py

nothing added to commit but untracked files present
(use "git add" to track)
```

What it means: The red filename means `hello.py` is untracked. Red does not mean it is broken.

Stage the file

```
git add .
```

```
No output appears.
The prompt returns.
```

What it means: `git add` chooses which changes Git will include in your next commit.
The dot `.` means every changed file in this repository.
To choose only `hello.py`**, use its path from the repository folder:**

```
git add ./hello.py
```

TERMINAL · SAVE AND PUBLISH

2. Verify and create a commit

Run each command with **Enter**. Your commit code may look different from this example.

Check that the file is staged

```
git status
```

```
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
  new file:   hello.py
```

What it means: The green filename means the file is staged. It will be included in your next commit.

Save it in Git's history

```
git commit -m "Printed hello"
```

```
[main a4058f0] Printed hello
 1 file changed, 1 insertion(+)
 create mode 100644 hello.py
```

What it means: Git created the commit locally. Your seven-character code will be different.

TERMINAL · SAVE AND PUBLISH

3. Check and push to GitHub

Press Enter after each command. Git may print slightly different numbers on your computer.

Check the local commit

```
git status
```

```
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

What it means: The commit exists on your computer, but has not reached GitHub yet.

Send the commit online

```
git push
```

```
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), done.
Total 3 (delta 0), reused 0 (delta 0)
To https://github.com/username/name-of-your-repo.git
   a1c79f4..a4058f0  main -> main
```

What it means: The progress lines are normal. `main → main` confirms the commit is now on GitHub.

EXERCISE · WORK FROM TOP TO BOTTOM

Make and publish a commit

1. **Change `hello.py`**
   Add a new line, for example:
   `print("I made a commit!")`

2. **Save the file**
   Use `Ctrl + S` on Windows or `⌘ + S` on macOS.

3. **Open the terminal**
   Choose *Terminal* → *New Terminal*.

Finished? Open your repository on GitHub and look for the new commit.

```
# 4. Check what changed
git status

# 5. Choose hello.py for the commit
git add ./hello.py

# 6. Check that hello.py is green
git status

# 7. Create the commit
git commit -m "Update hello"

# 8. Check that the commit is ready
git status

# 9. Send it to GitHub
git push
```

TERMINAL · DOWNLOAD A REPOSITORY

How do I get a repository from GitHub?

Choose where the repository should live before you download it.

1. **Open VS Code**
   Choose *File* → *Open Folder…*

2. **Choose the parent folder**
   Select *Documents*, or create a *Code* folder there.

3. **Open the terminal**
   Choose *Terminal* → *New Terminal*. It opens in the selected folder.

4. **Clone the repository**

```
git clone https://github.com/username/name-of-your-repo.git
```

5. **Open the new repository**
   Choose *File* → *Open Folder…* and select the `name-of-your-repo` folder.

Next time: get updates

Open the repository folder and its terminal, then run:

```
git pull
```

TERMINAL · CHEAT SHEET

Git Cheat Sheet

Run these commands one at a time. Press **Enter** after each command.

```
# Get a repository for the first time
git clone https://github.com/username/name-of-your-repo.git

# Get its newest changes later
git pull

# 1. Check what Git sees
git status

# 2. Choose hello.py for the next commit
git add ./hello.py

# 3. Check that hello.py was added
git status

# 4. Create the commit
git commit -m "Printed hello"

# 5. Check that the commit is ready to publish
git status

# 6. Send the commit to GitHub
git push
```

TAKE-HOME EXERCISE · PREREQUISITES

Read this folder structure

Study the tree, then answer the four questions.

```
my-project/
|-- hello.py
|-- notes.txt
`-- images/
    `-- logo.png
```

1. Name the **three files**.
2. Name the **two folders**.
3. What is the file extension of `hello.py`?
4. What path leads from `my-project` to `logo.png`?

SOLUTION · PREREQUISITES

Files and folders

1. **Files**
   `hello.py`, `notes.txt`, `logo.png`

2. **Folders**
   `my-project` and `images`

3. **File extension**
   `.py`

4. **Path**
   `my-project/images/logo.png`

TAKE-HOME EXERCISE · EXPLORE A PYTHON PROJECT

Clone and play a Python game

1. **Open a new VS Code window**
   Choose *File* → *New Window*.

2. **Open the terminal**
   Choose *Terminal* → *New Terminal*. Type and run every command shown on this slide there.

3. **Clone and explore**
   Open the new `PythonStdioGames` folder and find `bagels.py`.

Finished? Play Bagels: guess the secret three-digit number using the clues.

In the terminal, run:

```
git clone https://github.com/asweigart/PythonStdioGames.git
```

Open the new repository folder in VS Code. Open its terminal, then run:

```
git status
```

Run the game on Windows:

```
python src/gamesbyexample/bagels.py
```

On macOS:

```
python3 src/gamesbyexample/bagels.py
```

ADVANCED TAKE-HOME EXERCISE · TERMINAL

Build folders using commands

1. **Go home**
   Use `cd` to move to your home folder.

2. **Create and enter a folder**
   Make `terminal-practice`, then move into it.

3. **Build the structure**
   Create the folders shown on the right.

4. **Check your work**
   Use `ls` after each step.

Finished? Your final `ls` shows `projects` and `notes`.

Build this structure:

```
terminal-practice/
|-- notes/
`-- projects/
    `-- python/
```

Navigate between folders:

```
cd "$HOME"
cd folder-name
cd ..
```

Hint: Replace `folder-name` with the folder you want to create or enter.

```
mkdir folder-name
ls
```

`mkdir` creates a folder. `ls` lists what is in your working folder.

SOLUTION · ADVANCED TERMINAL EXERCISE

Build folders using commands

Run one command at a time:

```
cd "$HOME"
mkdir terminal-practice
cd terminal-practice
mkdir notes
mkdir projects
ls
cd projects
mkdir python
ls
cd ..
ls
```

The final output should include `notes` and `projects`.